

Comparison of Multiple GPU Scheduling Algorithms for FlashInfer Attention Inference

Francesco Carboni, Author 2, Author 3,
Delft University of Technology

ABSTRACT

Abstractly this is an ACM CCS Template.

1 INTRODUCTION

Tiling remains one of the most fundamental and effective techniques for maximizing inference efficiency in Large Language Models (LLMs). Crucially, the selection of tile size directly determines how effectively an execution engine saturates the underlying GPU hardware. Consequently, the micro-scheduling strategy, which governs how these structural tiles are distributed across the available Streaming Multiprocessors (SMs), becomes a primary lever for operational throughput.

Among modern work-decomposition methods, dynamic, run-time scheduling strategies derived from Stream-K[7] represent the state-of-the-art, whereas static, data-parallel partitioning schemes remain highly susceptible to quantization inefficiencies. For instance, FlashInfer[13] leverages a dynamic scheduling paradigm built upon the core tenets of Stream-K.

In this work, we isolate the impact of these tile-based workload scheduling strategies, comparing both Stream-K derivatives and standard baselines. We aim to provide a rigorous, algorithmic evaluation of their performance across varying input sequence length distributions. Furthermore, we formalize the dynamic workload distribution strategy of FlashInfer as a classic Multi-way Number Partitioning problem, exploring opportunities to optimize its execution grid mapping by evaluating well-known heuristic partition solvers.

2 BACKGROUND

Large Language Models have become central to a wide range of applications, from professional tools to everyday consumer products. These models are built on the Transformer architecture, whose core computational primitive is the attention mechanism [10, 15]. At a high level, attention computes a weighted combination of value vectors, where the weights are determined by the similarity between a query vector and a set of key vectors. The efficiency of this computation is therefore directly tied to the sizes of the query, key, and value matrices, making it a primary bottleneck in inference throughput and latency.

2.1 LLM Inference and the KV-Cache

Serving a Transformer-based model to a user involves two structurally distinct phases: prefill and decode. During the prefill phase, the model processes the entire input prompt in parallel, computing the key and value representations for every token in the sequence. These representations are stored in a dedicated memory structure called the KV-Cache, which persists across decoding steps to avoid redundant recomputation. During the decode phase, the model generates output tokens one at a time in an autoregressive loop

[5, 8, 14]. At each step, only the query vector of the newly generated token needs to be computed; the key and value vectors of all preceding tokens are read directly from the KV-Cache. This reduces the time complexity of each decoding step from quadratic to linear in the sequence length [9].

The KV-Cache is therefore not merely an optimization but a structural requirement for efficient autoregressive inference [12]. However, it introduces a significant memory management challenge: different requests in a batch have different prompt lengths and generate responses of different lengths, causing their KV-Caches to grow at different rates and reach different sizes.

2.2 Tiling

To mitigate the risk of having a huge bottleneck, one of the most important optimization of GPU computation is tiling. Tiling breaks large global tensors down into small, fixed-size submatrices designed to precisely fit the GPU's local shared memory boundaries[4, 11].

While this technique successfully keeps data movement within ultra-fast on-chip memory, classic data-parallel tiling struggles to maintain high processor utilization across modern, wide GPU architectures. Traditionally, deep learning libraries partition work by dividing the output matrix into regular, independent tiles assigned to concurrent thread groups[1, 2, 6]. These tiles execute across physical hardware cores in successive "waves". This approach introduces severe performance bottlenecks due to quantization inefficiency, which occurs when the total number of output tiles is not an exact, even multiple of the available physical processor cores. When the final execution wave is only partially full, the unused cores are forced to sit completely idle, wasting compute power.

To overcome these structural limitations, newer work decomposition strategies like Stream-K [7] abandon rigid, uniform output tiling. Instead of binding thread groups to static output boundaries, Stream-K linearizes the underlying matrix-multiplication loop and streams near-perfectly equal shares of the aggregate inner-loop iterations across the available physical cores. This work-centric parallelization effectively eliminates quantization inefficiency and provides consistent, near-peak hardware utilization across diverse matrix geometries.

2.3 GPU Memory Hierarchy

The physical organization of a GPU determines the performance characteristics of an attention operator based on where the data resides during execution.

Serving as the off-chip main memory pool, the global memory, backed up by High-Bandwidth Memory (HBM), stores massive tensors like the model weights and the multi-gigabyte Key-Value (KV) cache. During inference time, model weights and keys and values associated to the query, have to be transferred between the global

memory and the Streaming Multiprocessors (SMs) cache system, starting from the L2 level. Below this level, located directly inside each individual SM, the shared memory is an ultra-fast, cache pool that serves all the thread blocks in an SM. It provides exceptionally low latency but it is structurally constrained in size. Positioned at the very tip of the memory pyramid, registers provide immediate operand access to individual execution threads[3]

2.4 FlashInfer

2.5 Other Strategies

FlashAttention / FlashDecoding + LeanAttention

3 METHODOLOGY

Despite implementing the scheduling algorithms directly in a CUDA-compatible language and profiling them on real GPU hardware would yield the most accurate performance measurements, this would introduce a wide range of system-level design choices, such as kernel implementation, memory layout, CUDAGraph compatibility. That fall outside the scope of this work. Instead, we developed a lightweight simulator that isolates the scheduling problem from its hardware context, allowing us to focus strictly on algorithmic analysis.

3.1 Assumptions

In our work we assume that, when

3.2 Components

Our simulator takes as primary inputs the tile size (T_q) and the request size array (l_{kv} and l_q), producing an execution allocation map that distributes the workload across a fixed grid of Cooperative Thread Arrays (CTAs). Each scheduling algorithm partitions and dispatches these work items according to its own architectural logic.

To enforce hardware isolation, each virtual CTA is simulated as an independent Python Worker process explicitly bound to a dedicated physical CPU core via core affinity utilities, preventing operating system scheduling skew. To mimic the concurrent instruction execution of warps without the overhead of complex multi-threading libraries, the aggregate loop iteration volume assigned to a Worker is scaled down mathematically by a constant warp concurrency factor (32) and executed sequentially.

Workers track execution metrics across distinct simulated memory scopes: a local scratchpad array representing low-latency on-chip Shared Memory (SRAM), and a global data structure representing cold off-chip Global Memory (HBM). While reads within the local SRAM scope incur zero simulated time penalties, any mandatory data transfers from the un-cached global HBM data structure trigger a strict, hardcoded latency penalty (e.g., +200 ns) to realistically simulate processor memory-bandwidth stalls.

The execution loop simulates the arithmetic volume of the core attention matrix operators (QK^T and PV products) by incrementing a local iteration counter directly proportional to the assigned matrix tile boundaries ($l_q \times l_{kv}$). The simulator does not model instruction-level parallelism or physical cache line collisions as it would result

in an increase in complexity, which is outside of the scope of this project.

3.3 Evaluation Metrics and Cost Characterization

To analyze the performance of the evaluated scheduling algorithms independently of host-side execution noise, the simulation environment bypasses physical CPU wall-clock measurements. Instead, it tracks three deterministic cost metrics that serve as direct proxies for hardware efficiency, load distribution, and memory bottlenecks on physical accelerators.

3.3.1 Loop Iteration Count. The loop iteration count measures the absolute arithmetic workload assigned to each individual process. Inside the innermost execution loop of each Worker, an integer counter increments sequentially for every simulated multiply-accumulate (MAC) tile operation.

$$\text{Workload}_{\text{Worker}} = \sum \text{Iterations}$$

This metric is utilized to directly quantify load imbalance across the virtual hardware grid. For instance, when evaluating highly skewed batches containing irregular sequence lengths, computing the delta between the maximum and minimum iteration counts across the active Workers isolates the exact structural imbalance induced by static partitioning strategies like FlashDecoding [].

3.3.2 Simulated Latency. Because CPU core clock cycles do not match the massive parallel execution throughput of physical GPU SMs, tracking CPU physical execution time is inaccurate. Instead, in the simulator, each Worker implements a stateful time-accumulation model that simulates the real execution time, using a local floating-point variable ($\text{Latency}_{\text{Sim}}$) initialized to zero.

For every step executed in the arithmetic inner loop, the clock increments by a fixed operational step value ($\Delta t_{\text{math}} = 10 \text{ ns}$) representing the compute cost of an attention tile block. Furthermore, to capture the memory-bandwidth bottlenecks of the decode phase, the model checks if the required Key-Value page indices reside in the local SRAM scope. If a cache miss occurs, the clock is penalized by an additional memory transfer constant ($\Delta t_{\text{mem}} = 200 \text{ ns}$) to accurately simulate off-chip global memory retrieval time.

$$\text{Latency}_{\text{Sim}} = (\text{Iterations} \times \Delta t_{\text{math}}) + (\text{Cache Misses} \times \Delta t_{\text{mem}})$$

This explicit tracking allows the framework to evaluate how efficiently algorithms like FlashInfer schedule memory-bound tasks to prevent processing units from stalling during global data transfers.

3.3.3 Quantization Inefficiency. To capture the impact of quantization inefficiencies and straggler effects typical of wide execution grids, the simulator measures thread synchronization delays. Upon completing its assigned work array, each Worker logs its final accumulated simulated latency timestamp to a shared array visible to the master thread before entering a synchronization barrier ('multiprocessing.Barrier').

Once all active Workers clear the barrier, the synchronization slack for each individual core is computed as follows:

$$\text{Slack}_{Core} = \max(\text{Shared_Array}) - \text{Shared_Array}[Core]$$

The overall execution duration for the processing wave is defined by the maximum recorded latency in the shared array, $\max(\text{Shared_Array})$. High aggregate slack values across the worker grid indicate that faster cores are sitting idle while waiting for a heavily loaded straggler process to complete, providing a clear mathematical metric for quantization inefficiency.

4 RESULTS

Provide the outcome of your experiments and measurements.

5 DISCUSSION

This section will be part of the exam.

6 CONCLUSION

This section will be part of the exam.

REFERENCES

- [1] Ahmad Abdelfattah, David Keyes, and Hatem Ltaief. 2016. Kblas: An optimized library for dense matrix-vector multiplication on gpu accelerators. *ACM Transactions on Mathematical Software (TOMS)* 42, 3 (2016), 1–31.
- [2] Andrew Kerr, Duane Merrill, Julien Demouth, and John Tran. 2017. Cutlass: Fast linear algebra in cuda c++. *NVIDIA Developer Blog* 2 (2017), 10.
- [3] Philip Kiely. 2026. *Inference engineering*.
- [4] David B. Kirk and Wen-mei Hwu. 2010. *Programming massively parallel processors a hands-on approach*. <https://doi.org/10.1016/c2015-0-02431-5>
- [5] Baolin Li, Yankai Jiang, Vijay Gadepally, and Devesh Tiwari. 2024. Llm inference serving: Survey of recent advances and opportunities. In *2024 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 1–8.
- [6] Rajib Nath, Stanimire Tomov, and Jack Dongarra. 2010. An improved magma gemm for fermi graphics processing units. *The International Journal of High Performance Computing Applications* 24, 4 (2010), 511–515.
- [7] Muhammad Osama, Duane Merrill, Cris Cecka, Michael Garland, and John D Owens. 2023. Stream-k: Work-centric parallel decomposition for dense matrix-matrix multiplication on the gpu. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, 429–431.
- [8] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. 2018. Improving language understanding by generative pre-training. (2018).
- [9] Luohe Shi, Hongyi Zhang, Yao Yao, Zuchao Li, and Hai Zhao. 2024. Keep the cost down: A review on methods to optimize LLM’s KV-cache consumption. *arXiv preprint arXiv:2407.18003* (2024).
- [10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [11] Chang Xu, Steven R Kirk, and Samantha Jenkins. 2009. Tiling for performance tuning on different models of GPUs. In *2009 Second International Symposium on Information Science and Engineering*. IEEE, 500–504.
- [12] Yichun Xu, Navjot K Khaira, and Tejinder Singh. 2026. KV Cache Optimization Strategies for Scalable and Efficient LLM Inference. *arXiv preprint arXiv:2603.20397* (2026).
- [13] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, et al. 2025. Flashinfer: Efficient and customizable attention engine for llm inference serving. *Proceedings of Machine Learning and Systems* 7 (2025).
- [14] Zhihang Yuan, Yuzhang Shang, Yang Zhou, Zhen Dong, Zhe Zhou, Chenhao Xue, Bingzhe Wu, Zhikai Li, Qingyi Gu, Yong Jae Lee, et al. 2024. Llm inference unveiled: Survey and roofline model insights. *arXiv preprint arXiv:2402.16363* (2024).
- [15] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Zican Dong, Yupeng Hou, Beichen Zhang, Yingqian Min, Junjie Zhang, Peiyu Liu, et al. 2026. A survey of large language models. *Frontiers of Computer Science* 20, 12 (2026), 2012627.

A APPENDIX